Reporte técnico — Pipeline de observabilidad end-to-end con OpenTelemetry dual-cloud (AWS + GCP)

Laboratorio de observabilidad — entregable del repositorio `observabilidad-otel`

2026-10-15

Índice

1. Portada y resumen ejecutivo **2**

2. Arquitectura **2**
2.1. Diagrama dual-cloud . . . 2
2.2. Flujo de una request . . . 3
2.3. Topología del collector: Fargate y GKE . . . 3
2.4. Alternativas evaluadas, con su precio . . . 3

3. Instrumentación **4**

4. Configuración y despliegue del Collector **5**
4.1. Pipeline AWS (ADOT v0.46.0) — exporter a exporter . . . 5
4.2. Pipeline GCP (contrib 0.162.0 / GCP-distro) . . . 5
4.3. Versionado y validación en CI . . . 6

5. Correlación cross-signal **6**
5.1. Mecanismo . . . 6
5.2. La prueba: 13/13 en la unión de corridas . . . 6

6. Análisis de overhead **7**
6.1. Metodología . . . 7
6.2. Resultados . . . 7
6.3. Ruido, límites y lectura honesta . . . 7
6.4. Recomendaciones de tuning . . . 8

7. Presupuesto y control de coste **8**
7.1. Líneas base verificadas . . . 8
7.2. Guardarraíles implementados . . . 8
7.3. Coste real del mes (por proveedor, desde los CSVs de evidencia) . . . 9

8. Mapeo dual-cloud y desviaciones restantes **9**

9. Conclusiones y trabajo futuro **10**
Apéndice A — Índice de ADRs . . . 11

Apéndice B — Figuras (placeholders pendientes de captura humana) 11
Apéndice C — Reproducibilidad 11

1. Portada y resumen ejecutivo

Qué se construyó. Un pipeline de observabilidad *end-to-end* con OpenTelemetry sobre **dos nubes simultáneas**: dos microservicios FastAPI (`service-a` → `service-b` → PostgreSQL 16) instrumentados con el SDK de OTel, un **OTel Collector por nube** (ADOT v0.46.0 en AWS; contrib 0.162.0 en GCP/local) y los backends nativos de cada proveedor — X-Ray, CloudWatch Logs y EMF en AWS; Cloud Trace, Cloud Logging y Cloud Monitoring (+ Managed Prometheus y una Jaeger UI en-cluster) en GCP. El cómputo: **ECS Fargate** (AWS) y **GKE Standard** con pool spot (GCP). El mismo código, mismo binario y mismo seed corren en cuatro modos (local, AWS, GCP, dual-cloud); solo cambian el YAML del collector, las variables de entorno y el cómputo.

Resultado headline del overhead. Con la variante instrumentada en su configuración de diseño (sampling *parent-based* 0.25), la campaña ABBA local midió **+4,94 % de overhead en p99** sobre la línea base — cifra que el propio reporte declara **subestimada y contaminada por un outlier** de arranque (una réplica A con +708 ms de desviación). La lectura limpia de la misma campaña: **p50 +22,5 %, CPU +6,6 %, memoria +24,5 %, throughput −18 %** en bucle cerrado. El coste incremental de instrumentar fue **$0,1430 USD en AWS y $0,0000 en GCP** (ingesta real de logs), con X-Ray y Cloud Trace en **$0 demostrado** (trazas gestionadas apagadas durante el benchmark, decisión C2).

Correlación demostrada, no afirmada. El framework `verify_correlation.py` cerró **13/13 aserciones** (unión de corridas del 2026-10-12): las 9 del lado AWS en una corrida mono-nube con exit 0, las 3 del lado GCP en otra, y la aserción **cruzada A13** — el *mismo* `trace_id` W3C visible en X-Ray/ CloudWatch **y** en Cloud Trace — con `service-a` ejecutando en ECS y llamando al `service-b` desplegado en GKE a través de un LoadBalancer regional.

Veredicto de las decisiones de diseño. Las diez decisiones registradas en `docs/adr/` se sostienen: el sustrato de cómputo elegido en cada nube cabía en presupuesto con teardown verificable; las dos desviaciones forzadas por la realidad de la plataforma (no existe el port-forwarding de SSM a tasks Fargate; los exemplars no sobreviven al exporter Prometheus del collector) se resolvieron con sustitutos más simples que lo previsto y **verificados en vivo**. El presupuesto de créditos ($100 AWS + $300 GCP) no se agotó: el gasto acumulado registrado en las sesiones de nube es del orden de **$3,4 USD** en concepto de cómputo/ingesta medible, dentro de los guardarraíles ($1/día AWS; budget COP 964 461 en GCP; gate de 150 h de task).

2. Arquitectura

2.1. Diagrama dual-cloud

```
MODO DUAL (A13) -- el trace_id cruza la frontera de nubes:
  service-a (ECS Fargate, AWS)
     |  HTTP /inventory/reserve
     v  INVENTORY_URL -> LB regional (GCP)
  service-b (GKE Standard, GCP) -- SQL --> postgres (GKE)

MODO AWS (mono) -- una task Fargate awsvpc:
  service-a :8000 -> service-b :8001 -> postgres :5432 (localhost)
```

```
  collector ADOT v0.46.0 (mismo namespace de red):
     +- awsxray            -> X-Ray
     +- awsemf             -> CloudWatch EMF
     +- awscloudwatchlogs  -> CloudWatch Logs
     +- prometheus :8889   (scrape local)
  sidecar cloudwatch-agent -> metricas ECS

MODO GCP (mono) -- namespace otel-lab (Helm):
  service-a -> service-b :8001 -> postgres :5432 (ClusterIP)
  collector contrib 0.162.0:
     +- googlecloud             -> Cloud Trace
     |    (fan-out) otlphttp    -> Jaeger UI (pod en-cluster)
     +- googlemanagedprometheus -> Cloud Monitoring
     +- (fluent-bit/GKE)        -> Cloud Logging
     +- prometheus exporter :8889
```

La versión mantenida de este diagrama, con el detalle de cada pieza, está en `ARCHITECTURE.md`; las decisiones de topología y sus alternativas, en los ADRs enlazados abajo.

2.2. Flujo de una request

Un `POST /orders` con `traceparent` W3C inyectado por el cliente recorre: spans de negocio en service-a (`order.process` → `order.validate`, `inventory.reserve`, `payment.authorize`, `order.persist`), salto HTTP a service-b (`POST /inventory/reserve` → `inventory.query` → `SELECT/UPDATE inventory`; `payment.gateway_call`), y SQL en ambos lados. El mismo `trace_id` aparece en spans, en los logs JSON (structlog, inyectado por el SDK) y en los contadores/histogramas; los exemplars métrica→traza se sirven desde el scrape **de la aplicación** (`:8000/metrics`, OpenMetrics — §5.1 y ADR-007). En modo dual, el salto service-a→service-b cruza la frontera de nube y las señales quedan indexadas en los backends de **ambas** nubes.

2.3. Topología del collector: Fargate y GKE

- **AWS**: el collector es un contenedor más de la task (5 contenedores en 1 vCPU/2 GB; reservas 1024 CPU/1920 MB, `docs/aws-ecs.md` §3). El YAML va **horneado en la imagen** (Fargate no admite bind-mount); los secretos por SSM; la identidad de infra por el endpoint de metadatos de task (`resourcedetection/ecs`).
- **GCP**: el collector es un Deployment del chart `helm/otel-lab` con ConfigMap sincronizado byte-a-byte desde `deploy/collector/`; se suplantó al service account de GCP vía **Workload Identity** (cero claves JSON); el waterfall navegable se consigue con fan-out a una Jaeger all-in-one dentro del cluster (ADR-004/006).
- **Acceso humano sin ingress permanente**: regla /32 temporal con IP autodetectada en AWS (ADR-003 — el túnel SSM a Fargate **no existe**, verificado en vivo); `kubectl port-forward` autenticado en GCP. SG/firewall en steady state con **0 ingress**, verificado tras cada revoke.

2.4. Alternativas evaluadas, con su precio

Nube	Opción	Coste relevante	Veredicto
AWS	**ECS Fargate** (task 1 vCPU/2 GB)	aprox. **$0.04928/h** medido; $0 con desired_count=0	**Elegida** (requisito del enunciado; ADR-002)

Nube	Opción	Coste relevante	Veredicto
AWS	EC2 sobre ECS	nodo 24/7 aunque la sesión duerma	Descartada
AWS	EKS	control plane + complejidad	**Vetada** (lista negra)
AWS	ALB + HTTPS público	~$0.0225/h + datos	**Vetada**; acceso por /32 temporal (ADR-003)
AWS	NAT Gateway	~$0.045/h + GB procesados	**Vetada**; egress por IGW con IP efímera
GCP	**GKE Standard zonal**	fee plano **$0.10/h** (free tier cubre 1 cluster zonal) + spot	**Elegida** (ADR-004)
GCP	GKE Autopilot	reserva por pod 24/7, sin spot	**Vetada** (lista negra)
GCP	Cloud Run	min-instances>0 facturable	**Vetada**
GCP	Cloud NAT	~$0.0475/h + GB	**Vetada**; Private Google Access ($0)
GCP	Cloud LB gestionado permanente	regla de forwarding + datos	**Vetada**; el LB regional solo vive en el modo dual (<$0.01 de datos en el lab)
GCP	nodo e2-small (plan original)	~$0.006/h spot	e2-small no cabe en GKE 1.36 (system pods ~856m); e2-medium ~$0.012/h — ADR-005

Figuras de esta sección (placeholders pendientes de captura humana; el índice está en la carpeta docs/evidence/img): Figura A5 — task Fargate con 5 contenedores (fig-a5-ecs-service-tasks); Figura G5 — workloads GKE (fig-g5-gke-workloads).

3. Instrumentación

- **Un único motor por servicio** (`app/telemetry.py`): registra auto- instrumentación FastAPI/uvicorn/httpx/SQLAlchemy/psycopg2/logging, crea el SDK (trazas, métricas, logs) y es la única superficie de configuración. El toggle `OTEL_ENABLED=false` no deja ningún exporter activo — es la base del A/B honesto del benchmark.
- **Propagadores**: W3C TraceContext (canónico) + X-Ray (`opentelemetry-propagator-aws`) + `X-Cloud-Trace-Context` (`propagator-gcp`) en compuesto, para que el contexto viaje por ambas nubes (PLAN L40).
- **Spans custom de negocio** (no solo HTTP): `order.process`, `order.validate`, `order.persist`, `inventory.reserve`, `payment.authorize`, `payment.gateway_call`, `inventory.query` — visibles en el waterfall de Jaeger en GKE (traza real: 23 spans, raíz `POST /orders` aprox. 250 ms, `docs/evidence/04-jaeger-gcp.md`).
- **Driver PostgreSQL**: `psycopg2-binary==2.9.13` con SQLAlchemy 2.0 (ADR-001); los spans SQL salen por instrumentación, no a mano.
- **Sampling**: `parentbased_traceidratio` env-configurable — 1.0 para correlación/desarrollo, **0.25 en el benchmark** (ADR-009), con 1.0/0.0 reservados para los runs de atribución.
- **Logs**: structlog JSON con `trace_id`/`span_id` inyectados por el SDK; `access_log` de uvicorn desactivado; PII redactada también en el collector (`attributes/redact`).
- **Cardinalidad**: `order.id`, `user` y `trace_id` nunca como label de métrica (cubierto por `test_cardinality.py`); el dashboard de 6 paneles lee de Prometheus porque `GetMetricData` de CloudWatch se cobra siempre (decisión C1).
- **Exemplars**: el SDK los emite, pero el puente métrica→traza se sirve por la vía app-level (histograma `http_server_duration_milliseconds` con `exemplar={"trace_id","span_id"}` vía OpenMetrics) — ADR-007, §5.1.

4. Configuración y despliegue del Collector

4.1. Pipeline AWS (ADOT v0.46.0) — exporter a exporter

Pieza	Justificación	Coste por nube
receiver `otlp` (gRPC :4317) + `prometheus` (autotelemetría :8888)	entrada estándar del SDK + scrape propio	$0
`memory_limiter → resourcedetection(ecs) → resource → attributes/redact → batch`	protección de memoria de la task; `cloud.provider` / `deployment.environment` normalizados; PII fuera; amortización de red OTLP	$0 (CPU dentro del perfil de task)
`awsxray`	trazas gestionadas AWS; **apagado en benchmark** (C2)	$0 en campaña (demostrado); free tier 100k trazas/mes
`awsemf` con `filter/cloudwatch-metrics` (whitelist **4** contadores) + `NoDimensionRollup`	free tier de métricas custom (10); el rollup implícito generaba **26 variantes** por contador → 1	aprox.$0 dentro de free tier
`awscloudwatchlogs`	logs a grupos `/otel/labs/*` con retención **3 días**	$0.50/GB ingest — acotado por C3 (~0.4 KB/request)
`prometheus` (`:8889`)	el hub de correlación local lee de aquí	$0
`prometheusremotewrite` / `otlphttp/tempo` / `otlphttp/loki` (Grafana Cloud)	opcional **aplazada** (local-only); placeholders example.com fallando a propósito	$0 (no contratado)
sidecar `cloudwatch-agent` (`essential:false`)	métricas ECS — ADOT v0.46.0 no trae `hostmetrics` / `transform` (desviación R-09)	aprox.$0 (métricas de plataforma no tarifadas)

Validación: ADOT no tiene subcomando `validate` → se valida **por arranque limpio** ("Everything is ready…"); contrib sí → `validate`. Ambos binarios, en `make validate-collector` y en CI (ADR-006).

4.2. Pipeline GCP (contrib 0.162.0 / GCP-distro)

Pieza	Justificación	Coste por nube
`googlecloud` (trazas+logs+métricas)	Cloud Trace **preserva el ID W3C** (sin traducción tipo X-Ray); Cloud Logging con `jsonPayload.trace_id`; Cloud Monitoring	free allotments: 2,5M spans/mes; 50 GiB logs/mes
`googlemanagedprometheus`	series Prometheus gestionadas para el dashboard de 6 paneles GCP	$0.06/M samples, sin allotment — volumen del lab aprox. $0
fan-out `otlphttp/jaeger-gcp`	Jaeger UI navegable **en-cluster** (`docs/evidence/04-jaeger-gcp.md`)	nodo spot (marginal)
`resourcedetection(gcp)` + `k8s.*`	resource attributes que los backends de Google indexan	$0
pipeline de logs OTLP + fluent-bit/GKE	doble vía: los logs de sistema van por el agente; el pipeline OTLP quedó probado con `sent_log_records=130` tras el hotfix filelog	dentro de free tier
`batch` (métricas 60 s)	el batch corto provocaba drops por sampling period de GCM; 60 s lo resolvió	$0

Workload Identity / IAM: el SA del collector (`otel-lab-collector`, K8s) se suplantó al de GCP (`otel-lab-deploy`) con binding `roles/iam.workloadIdentityUser` — evidencia dura en `docs/evidence/01-wi-binding-gcp.md`, sobrevive al teardown. Roles least- privilege (cloud-trace.agent, logWriter, metricWriter…). Secreto `pg-password` montado por CSI Secret Manager **habilitado en el create** (ADR-005: el update post-create cuesta ~40 min de `UPGRADE_NODES`).

4.3. Versionado y validación en CI

Cuatro YAMLs versionados (`otel-collector-{local,aws,gcp,bench}.yaml`) con tags exactos de imagen (nunca `:latest`); el binario correcto valida cada config; la copia de auditoría del YAML aplicado se archiva en S3/GCS con clave = SHA-256 (paridad entre el chart, el deploy y el bucket: hash verificado byte a byte en la resesión del 2026-10-11).

5. Correlación cross-signal

5.1. Mecanismo

Pivote: el `trace_id` W3C (32 hex). Tres vías de enlace:

1. **Derived field + data link (Grafana)**: el regex `"trace_id":"([a-f0-9]{32})"` sobre los logs crea el enlace log→traza; los data links del dashboard enlazan panel→traza.
2. **Exemplars**: métrica→traza. Hallazgo clave (ADR-007): el exporter `prometheus` del collector **no propaga exemplars** (probado en ADOT 0.46 y contrib 0.162, con y sin OpenMetrics/timestamps; 0 exemplars en 1201 series en vivo); el cierre vive **en la app**: histograma `prometheus_client` nativo con `exemplar={"trace_id","span_id"}` del span activo, servido por negociación `Accept: application/openmetrics-text` (formato inline real, validado contra `BatchGetTraces` en X-Ray).
3. **El gotcha del ID en X-Ray**: el traductor OTLP→X-Ray reescribe a formato `1-8hex-24hex`; la igualdad literal con el traceparent es **imposible** → A3/A4 se verifican **por estructura** (grafo de servicio y parent_id encadenado), no por string. En Cloud Trace los IDs se preservan literales (A10 por igualdad directa).

5.2. La prueba: 13/13 en la unión de corridas

#	Aserción	Resultado	Corrida
A1	traceparent W3C inyectado en POST /orders	**PASS**	corrA8
A2	header `x-trace-id` de respuesta == trace	**PASS**	corrA8
A3	X-Ray: segmentos a/b con parent correcto (por estructura)	**PASS**	corrA8
A4	X-Ray: `db.system=postgresql` en segmento de service-b	**PASS**	corrA8
A5	CloudWatch Logs: >=1 entrada con trace_id por servicio	**PASS**	corrA8
A6	CloudWatch: `orders_created_total` incrementado (EMF, delta)	**PASS**	corrA8
A7	consistencia de resource traza<->log<->métrica	**PASS**	corrA8
A8	exemplars con trace_id de traza existente (scrape app-level)	**PASS**	corrA8
A9	límites de cardinalidad	**PASS**	corrA8
A10	Cloud Trace: traza W3C con spans de a y b (23 spans)	**PASS**	corrGCP6
A11	Cloud Logging: >=1 entrada con trace_id por servicio	**PASS**	corrGCP6

#	Aserción	Resultado	Corrida
A12	Cloud Monitoring: contador incrementado en la ventana	**PASS**	corrGCP6
A13	**cruzada GCP<->AWS: mismo trace_id en señales de ambas nubes**	**PASS**	corrDUAL3

`corrA8` (2026-10-12T20:33Z, task ECS `149e6bce…`, imágenes 0.1.4) salió con **exit code 0**; la corrida dual (`6cbbd62a…`) ejecutó service-a en ECS contra el service-b de GKE vía LB regional. En la corrida dual, A3-A7/A10-A12 degradan **estructuralmente** (cada nube solo ve la parte del path que le corresponde) — el cierre de fase es la unión documentada, no una corrida única. Evidencia: `docs/evidence/03-correlation.md` + 4 JSONs.

Figuras: A2 (X-Ray trace detail), A3 (CloudWatch logs), G1 (Cloud Trace waterfall), G2 (Cloud Logging), G4 (Jaeger en GKE) — placeholders en `docs/evidence/img/`.

6. Análisis de overhead

6.1. Metodología

Campaña **ABBA** contra el stack local (compose), k6 en Docker (50→100 VU, warmup + 5 min steady), 3 réplicas por variante, precalentamiento 60 s, cooldown 120 s, `down -v` entre runs, **mismo digest de imagen** A/B, seed idéntico; jaeger/grafana apagados, Prometheus activo como instrumento. Variante A: `OTEL_ENABLED=false`. Variante B: `true` + sampling 0.25 + pipeline de trazas del collector en sink `nop` (C2). Runs de atribución con sampling 1.0 y 0.0. Coste por nube leído del **ingest real de logs** de B con las compuertas `OTEL_BENCH_{AWS,GCP}_LOGS` abiertas — sin sesión de cómputo.

6.2. Resultados

Métrica	A (media)	B (media)	overhead
p99 latencia	1750,74 ms	1837,20 ms	**+4,94 %**
p50 latencia	aprox.968,6 ms	aprox.1186,7 ms	**+22,5 %**
CPU host (% no-idle)	12,45 %	13,27 %	+6,62 %
Memoria usada	1453,1 MB	1809,4 MB	**+24,52 %**
Throughput (bucle cerrado)	96,2 req/s	78,6 req/s	−18,3 %
Error rate	0,00 %	0,06 %	—

Atribución (p99): sampling 1.0 = 1788 ms; sampling 0.0 = 1617 ms; A = 1751 ms → generación+export de spans aprox. **9,75 %** del p99 base; "contexto/propagación" sale **−7,62 %** — **artefacto del outlier, no un resultado**.

6.3. Ruido, límites y lectura honesta

- **Outlier declarado**: A1 (primer run) = 2561 ms vs A2/A3 = 1249/1442 ms; desviación entre réplicas de A (708 ms) **mayor que el overhead que se quiere medir** → el +4,94 % sobre la media de p99 queda **subestimado**; la señal limpia está en p50 (+22,5 %). El p99 de B es *menor* que A1 porque B generó ~15 % menos carga: en bucle cerrado el overhead se manifiesta también como caída de throughput.

- **Perfil local != perfil nube**: collector + 2 servicios + postgres comparten el host del portátil; en Fargate (1 vCPU) la instrumentación compite con la app por el mismo core y el p99 real sería **peor**. Declarado como cota, no como medida de producción.
- **k6 comparte host con lo medido** (infla colas absolutas, se cancela en la diferencial).
- Sampling 0.0 no elimina el 100 % del coste de trazas (SDK sigue creando contexto y propagadores); ventanas de Prometheus aproximadas al minuto.
- **Salud del pipeline**: 0-2 batches en cola, 0 refusos/fallos en los 5 runs B — el batch 1024/5 s absorbe sin descartar señales.
- Discusión (por qué trazas > métricas, batching, sampling como palanca dominante, dimensionamiento): `docs/evidence/04-overhead.md`.

6.4. Recomendaciones de tuning

1. Bajar el sampling en producción (0.25 demostró ser la palanca dominante); usar 0.0 solo con conciencia de que contexto/propagadores siguen activos.
2. En perfiles 1 vCPU, separar el collector de la task (o subir memoria): el +24,5 % de RAM es lo más cerca del ceiling (límite 2 GB del perfil).
3. Mantener `batch` y export asíncrono: el overhead medido es CPU de serialización, no latencia de red.

Coste incremental de la campaña: AWS $0,1430 (5 runs B, 72-86k registros/ run, $0.50/GB); GCP $0,0000 (15-72 MB/run dentro de los 50 GB/mes libres); X-Ray/Cloud Trace **$0 demostrado**. Campaña total < $0,50.

7. Presupuesto y control de coste

7.1. Líneas base verificadas

- **AWS** (`docs/evidence/00-aws-baseline.md`, 2026-10-11): créditos $100 (dato declarado — no hay API de saldo); 16 recursos de lista blanca auditados contra free tier; budgets `free-tier` $100/mes + `otel-lab-daily` $1/día con alertas 80/100 %; Cost Anomaly Monitor `otel-lab-anomaly`; IAM least-privilege verificado por sondeo comportamental. El "sandbox" que bloqueó la Fase 0 resultó ser **SCP del propio organization** (region-lock + denial ecs/ecr/xray/ec2/s3) separada de la Raíz — no un caso de soporte.
- **GCP** (`docs/evidence/00-free-tier-baseline-gcp.md`): créditos $300 con ventana de 90 días (31 restantes a la fecha, dato declarado); cuotas reales leídas por CLI (Cloud Trace 3M spans/día; Logging 4,8 GB/min us-central1; Monitoring 30k req/min…); budget **COP 964 461** con umbrales **50/80/90/100/150 %** y `EXCLUDE_ALL_CREDITS` (ADR-008: sin excluir créditos la alerta queda ciega; la corrección fue vía API tras el error de moneda).
- **Billing consolidado AWS**: la cuenta del lab es **miembro**; el gasto va a la pagadora, que concentra ~$120 de créditos promocionales. Los budgets por-cuenta son proxy; el techo real es el pool del pagador (paso humano pendiente: budget payer-level).

7.2. Guardarraíles implementados

Guardarrail	Mecanismo	Verificación
Lista negra por proveedor	`guard_cost.py --provider aws|google` sobre el JSON del plan	en pre-commit, en CI de ambos roots y **antes del apply** en el workflow `deploy`

Guardarrail	Mecanismo	Verificación
Techo diario AWS	budget $1/día (IaC + CLI) con alertas 80/100 % + SNS	`describe-budgets` volcado en evidencia
Techo GCP	budget COP 964 461, umbrales 50-150 %	`billing budgets describe` en evidencia
Horas de task Fargate	gate de **150 h/mes** (`make budget-hours` + `instance-hours.csv`)	28,21 h acumuladas / 150 h al cierre de IaC; exit 0
Gasto diario en vivo	`make cost-guard` (Cost Explorer + barrido EC2/EIP/NAT/ALB/RDS y GCP)	tolerante a AccessDenied; reportado como "NO verificados"
Anomalías	Cost Anomaly Monitor por servicio + topic SNS `otel-lab-alerts`	cableado en IaC
Higiene	retención logs 3 d, lifecycle evidencia 7 d, ECR/AR keep-last-5, sin `prevent_destroy`	aplicado en IaC y CLI
Precondition de región	`precondition` (aborta) + `check` (avisa) en ambos roots	plan en us-west-2 aborta (verificado)
Teardown DoD	`aws-destroy/gcp-destroy` con verificación + registro CSV	todas las sesiones cerradas con lista vacía

7.3. Coste real del mes (por proveedor, desde los CSVs de evidencia)

Concepto	AWS	GCP
Cómputo de sesiones	$0,672 (13,55 h de task Fargate acumuladas)	$2,68 de fee de plano (26,82 h × $0,10) + nodo spot marginal (~$0,17-0,20 en las sesiones largas)
Ingesta de señales (bench + correlación)	$0,143 (logs; trazas $0)	$0,000 (logs dentro de free tier; trazas $0)
Sesión de correlación dual (12-oct)	~$0,67 (incluye pausa de 9,2 h con task activa)	~$1,6 (incluye la misma pausa facturando fee de plano)
Total medido en concepto comparable	**aprox. $0,82**	**aprox. $2,9**

Disclosure declarado: la sesión de correlación cerró en ~**$2,27 > 1 USD** (criterio de aceptación superado): el driver fue la pausa de 9,2 h con nubes encendidas — `gcp-stop` escala el nodo pero **no** detiene el fee de plano. La lección quedó fijada en el registro: *delete entre bloques, no stop*. No es arreglable retroactivamente; se declara para que la validación humana fuera informada.

Figuras: A6 (AWS Budgets + anomaly), G6 (GCP budget), A4 (dashboard EMF).

8. Mapeo dual-cloud y desviaciones restantes

Concepto	AWS	GCP	Nota
Cómputo	ECS Fargate (task única)	GKE Standard (pool spot)	perfiles 1 vCPU/2 GB <-> e2-medium
Registro de imágenes	ECR (IMMUTABLE, keep-last-5)	Artifact Registry (cleanup 7 d/keep-5)	paridad exacta
Secretos	SSM Parameter Store (Standard)	Secret Manager + CSI	nunca Secrets Manager (lista negra) <-> nunca claves JSON

Concepto	AWS	GCP	Nota
Identidad del workload	task role + `resourcedetection` (ecs)	Workload Identity + `resourcedetection` (gcp)	paridad funcional
Trazas	X-Ray (ID traducido → verificación estructural)	Cloud Trace (ID W3C preservado)	**desviación mínima inevitable**
Logs	CloudWatch Logs (exporter OTel)	Cloud Logging (fluent-bit + OTLP)	doble vía en GCP
Métricas	EMF (whitelist 4 contadores)	Cloud Monitoring + Managed Prometheus	GMP sin allotment free (volumen aprox. $0)
Dashboard nativo	CloudWatch (3 widgets)	Cloud Monitoring (6 paneles mosaic)	p99→avg como proxy declarado en GCP
Acceso humano	/32 temporal (SSM port-forward **no existe**)	port-forward autenticado	ambos sin ingress permanente
Entrada pública	ninguna (IGW + IP efímera)	LB regional solo en modo dual	asimetría admitida: el cruce AWS→GCP necesita un destino
Budget	$1/día + $100/mes + anomaly + SNS	COP 964 461, 50-150 %, EXCLUDE_ALL_CREDITS	monedas distintas por diseño
State IaC	S3 + locking nativo (`use_lockfile`)	GCS + versioning (sin bucket de locks)	workspaces `lab|bench` en ambos

Desviaciones restantes, todas declaradas con su evidencia: (1) traducción de ID en X-Ray (ADR implícito en §5.1, A3/A4 por estructura); (2) perfil de nodo e2-medium != e2-small del plan (ADR-005, mitigación R-08 prevista); (3) hub Grafana **local-only** — Grafana Cloud aplazada con decisión de sesión, sus exporters quedan como placeholders fallando a propósito; (4) `db_client_operation_duration` no exportada como histograma en GCP (proxy por endpoint de service-b, anotado en la validación del dashboard); (5) bucket de state GCP **sin lifecycle** a propósito (borrar el state rompería locking/auditabilidad); (6) las 14 figuras del reporte son **placeholders** pendientes de captura humana (índice en `docs/evidence/img/README.md`).

9. Conclusiones y trabajo futuro

Conclusiones. (1) El pipeline dual-cloud funciona: tres señales, dos nubes, correlación **demostrada** con 13/13 aserciones y un `trace_id` que cruza fronteras — no es una afirmación de diseño sino una corrida con exit 0 registrada. (2) El coste es un **requisito de diseño gobernable**: operar dentro de créditos con guardarraíles en capas funcionó; el único susto (la pausa de 9,2 h) se convirtió en regla operativa verificable. (3) Las plataformas mienten en los docs y dicen la verdad en la API: el túnel SSM a Fargate no existe, los exemplars no cruzan el collector, ADOT no valida, `gridWidgets` no se acepta, GKE 1.36 no cabe en e2-small — cada una de esas sorpresas generó un ADR y un sustituto más simple. (4) El overhead de instrumentar con OTel es real y abaratable (~+22 % en p50, ~$0,14 por campaña de 5 runs) con sampling como palanca dominante.

Trabajo futuro. Para convertir el lab en producción: relajar HA (cluster regional GCP / multi-AZ ECS con ALB — hoy vetados por coste), subir los perfiles de cómputo, re-encender Grafana Cloud (los exporters ya existen y están gateados), migrar psycopg2→psycopg3, medir el overhead **en** los perfiles nube (la cota local está declarada; Fargate 1 vCPU sería peor), y completar las 14 capturas humanas. La multi-nube **ya es el diseño**, no trabajo futuro.

Apéndice A — Índice de ADRs

Todos en `docs/adr/`: 001 driver psycopg2 · **002** cómputo AWS Fargate (proveedor AWS) · **003** acceso sin túnel SSM (proveedor AWS) · **004** GKE Standard spot (proveedor GCP) · **005** perfil de nodo e2-medium (proveedor GCP) · **006** collector dual ADOT/contrib · **007** exemplars app-level · **008** modelo de coste créditos+guardarraíles · **009** sampling 0.25 y trazas OFF · **010** dashboards nativos bajo límites de API.

Apéndice B — Figuras (placeholders pendientes de captura humana)

Este reporte no incluye capturas porque el agente no captura pantallas: cada figura está **reservada** en `docs/evidence/img/` con título, caption e instrucciones. El humano sustituye el `.svg` por el `.png` real (mismo nombre base) y regenera con `make report`.

ID	Fichero	Título
L1	`fig-l1-jaeger-local-waterfall`	Jaeger UI (local) — waterfall a→b→DB
L2	`fig-l2-grafana-slo-6-paneles`	Grafana OSS — dashboard SLO 6 paneles
A1	`fig-a1-xray-service-map`	X-Ray — service map
A2	`fig-a2-xray-trace-detail`	X-Ray — detalle de traza
A3	`fig-a3-cloudwatch-logs-trace-id`	CloudWatch Logs — trace_id por servicio
A4	`fig-a4-cloudwatch-emf-dashboard`	CloudWatch — dashboard EMF 3 widgets
A5	`fig-a5-ecs-service-tasks`	ECS Fargate — task con 5 contenedores
A6	`fig-a6-budgets-anomaly-alerts`	AWS Budgets + anomaly monitor
G1	`fig-g1-cloud-trace-waterfall`	Cloud Trace — waterfall W3C
G2	`fig-g2-cloud-logging-trace-id`	Cloud Logging — jsonPayload.trace_id
G3	`fig-g3-cloud-monitoring-dashboard`	Cloud Monitoring — dashboard 6 paneles
G4	`fig-g4-jaeger-gke-waterfall`	Jaeger UI en GKE — fan-out del collector
G5	`fig-g5-gke-workloads`	GKE — workloads del release otel-lab
G6	`fig-g6-gcp-budget-alerts`	GCP Budget — EXCLUDE_ALL_CREDITS

Apéndice C — Reproducibilidad

- Ejecución de los 4 modos: `README.md` (manual único). Modelo de coste y apagado: `COST.md`. Arquitectura de referencia: `ARCHITECTURE.md`.
- Evidencia cruda por fase: `docs/evidence/` (línea base 00, sesiones 01-02, correlación 03, overhead 04, dashboards 05, CSVs de horas).
- Este PDF: fuente `docs/report/report.md`; regeneración `make report` (pandoc + XeLaTeX en Docker, tag fijado). Verás el diff del PDF a través del diff del markdown.